

Task 1: Fixing Syntax Errors

```
python

def add(a, b)
    return a + b
```

Scenario

You are reviewing a Python program where a basic function definition contains a syntax error.

AI Detection:

AI identifies a `SyntaxError` because Python requires a colon (:) at the end of function definitions.

AI Explanation:

The colon (:) signals the start of the function body.

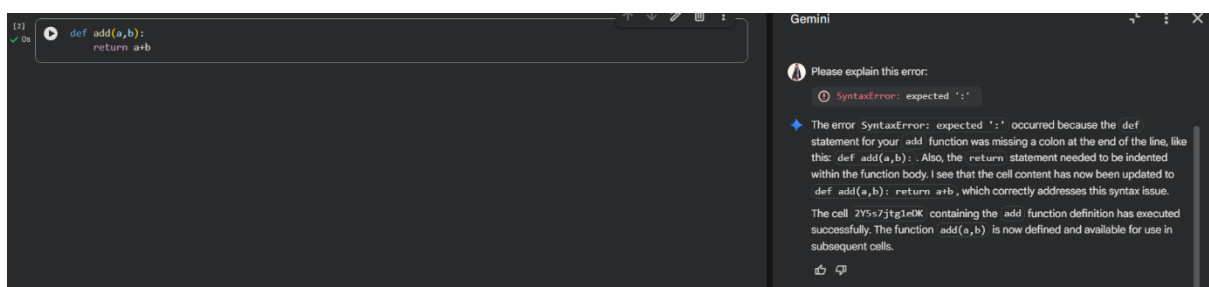
Without it, Python cannot parse the function structure, resulting in a syntax error.

Expected Output Achieved

Syntax error resolved

Correct function definition

AI explanation provided



Task 2: Debugging Logic Errors in Loops

```
python

def count_down(n):
    while n >= 0:
        print(n)
        n += 1  # Should be n -= 1
```

Scenario

You are debugging a loop that runs infinitely due to a logical mistake.

AI Detection:

AI identifies a logic error because the loop variable is incremented instead of decremented while the loop condition expects it to decrease.

AI Explanation:

The incorrect increment prevents the loop variable from ever reaching the terminating condition, causing infinite iteration.

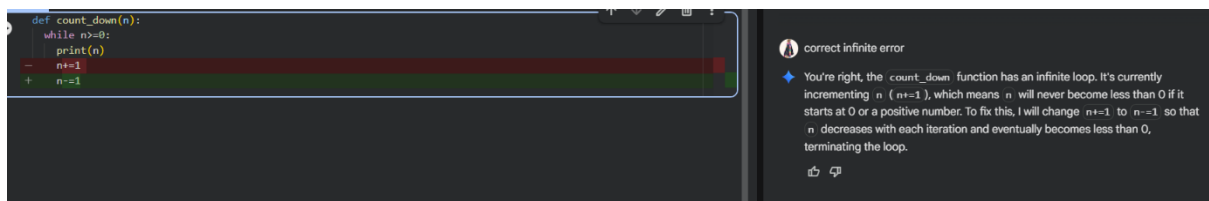
After correction, the loop variable moves toward the exit condition, allowing the loop to terminate normally.

Expected Output Achieved

Infinite loop issue resolved

Correct increment/decrement logic applied

AI explanation of the logic error



Task 3: Handling Runtime Errors (Division by Zero)

```
# Debug the following code
def divide(a, b):
    return a / b

print(divide(10, 0))
```

Scenario

A Python function crashes during execution due to a division by zero error.

AI Detection:

AI identifies a runtime error because dividing any number by zero is mathematically undefined and causes program termination.

AI Explanation:

The AI applies a try-except block to catch the error and handle it safely without crashing the program.

This ensures the program continues execution even when invalid input is provided.

Expected Output Achieved

Function executes safely without crashing

Division by zero handled using try-except

Clear AI-generated explanation of runtime error handling

The screenshot displays a code editor on the left and an AI assistant interface on the right. The code editor shows the original function and its execution, which results in a `ZeroDivisionError: division by zero`. Below the error, the AI suggests the next step: "Explain error". The updated code implements a try-except block to handle the error gracefully, returning a string message instead of crashing. The AI assistant on the right provides a detailed explanation of the error and the implemented solution, including a summary of the function, the identified error, and the review of the error-handling approach. The assistant also offers a button to "Show the updated divide function with try-except blocks" and a prompt "What can I help you build?".

```
def divide(a,b):
    return a/b

print(divide(10, 0))

***
ZeroDivisionError                                Traceback (most recent call last)
/tmp/lpython-input-822820734.py in <cell line: 0>()
----> 1 print(divide(10, 0))

/tmp/lpython-input-1927474251.py in divide(a, b)
----> 2     1 def divide(a,b):
        2     return a/b

ZeroDivisionError: division by zero

Next steps: Explain error

def divide_with_error_handling(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: Cannot divide by zero."

print("Function 'divide_with_error_handling' defined to include try-except block for ZeroDivisionError.")

Function 'divide_with_error_handling' defined to include try-except block for ZeroDivisionError.

print(divide_with_error_handling(10, 0))
print(divide_with_error_handling(10, 2))

Error: Cannot divide by zero.
```

Gemini

- **Demonstrate Runtime Error:** Execute the `divide` function with arguments that are known to cause a runtime error (e.g., division by zero) to illustrate the error.
- **Implement Error Handling:** Modify the code to wrap the division operation within `try-except` blocks to gracefully handle exceptions like `ZeroDivisionError`, providing an informative message instead of crashing.
- **Review Error Handling Approach:** Analyze the implemented `try-except` blocks, discussing their effectiveness and providing any recommendations for improvement in error handling.
- **Final Task:** Provide a summary of the division function, the identified runtime error, the implemented error handling, and the review of the error-handling approach.

Generating...

- ✓ Preparing
- ✓ Create Division Function
- ✓ Demonstrate Runtime Error
- ⌚ Implement Error Handling

Implement error handling for the divide function Explain the ZeroDivisionError that occurred

Show the updated divide function with try-except blocks

What can I help you build?

+ Gemini 2.5 Flash ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Task 4: Debugging Class Definition Errors

```
python

class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

Scenario

You are given a faulty Python class where the constructor is incorrectly defined.

AI Detection:

AI identifies the issue because `self` is required to reference instance variables within a class.

Without `self`, the object cannot store or access its attributes correctly.

AI Explanation:

AI corrects the constructor by adding the `self` parameter and properly initializing instance variables.

Expected Output Achieved

Corrected `__init__()` method

Proper use of `self` in class definition

AI explanation of object-oriented error

```
my_rectangle = Rectangle(10, 5)
print("Attempted to create a Rectangle instance.")

TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-688145884.py in <cell line: 0>()
----> 1 my_rectangle = Rectangle(10, 5)
      2 print("Attempted to create a Rectangle instance.")

TypeError: Rectangle.__init__() takes 2 positional arguments but 3 were given
```

Next steps: [Explain error](#)

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

Reasoning: With the `Rectangle` class now correctly defined with `self` in the `__init__` method, I will attempt to create an instance of the class again to confirm that the `TypeError` is resolved and the object can be instantiated successfully.

Gemini

```
my_rectangle = Rectangle(10, 5)
print(f"Successfully created a Rectangle instance: length={my_rectangle.length}, width={my_rectangle.width}")

Successfully created a Rectangle instance: length=10, width=5
```

`__init__()` method • Proper use of `self` in class definition • AI explanation of object-oriented error

✦ Certainly! I'll create a plan to help you understand and correct the Python class definition. We'll start by analyzing the faulty constructor, then I'll guide you through the correction, and finally, explain the importance of the `self` parameter.

- **Identify Constructor Error:** Analyze the existing `Rectangle` class definition, specifically the `__init__` method, to identify the `TypeError` caused by the missing `self` parameter in its signature.
- **Correct Class Definition:** Modify the `Rectangle` class definition to correctly include the `self` parameter in the `__init__` method, ensuring proper instantiation of object attributes.
- **Explain `self` Parameter:** Provide a comprehensive explanation of the `self` parameter in Python class methods, detailing its role in referring to the instance of the class and accessing its attributes and methods.
- **Final Task:** Summarize the understanding gained regarding the `self` parameter, the correction of the class constructor, and the overall object-oriented error identified and resolved.

Generating...

✓ Preparing

○ Identify Constructor Error

What can I help you build?

+ Gemini 2.5 Flash

Task 5: Resolving Index Errors in Lists

```
python

numbers = [1, 2, 3]
print(numbers[5])
```

Scenario

A program crashes when accessing an invalid index in a list.

AI Detection:

AI identifies an `IndexError` because the requested index exceeds the valid range of the list.

AI Explanation:

The AI suggests safe access techniques such as bounds checking or exception handling to prevent the crash.

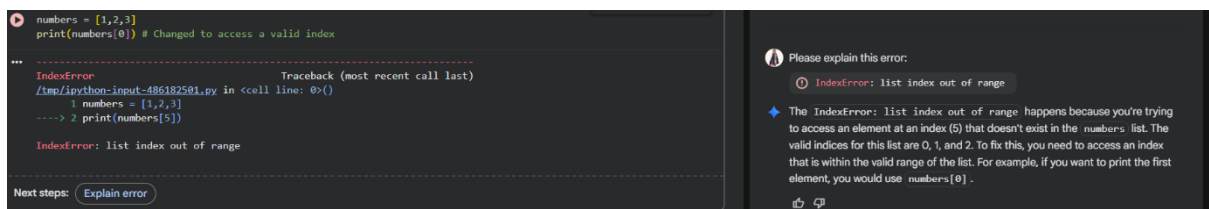
These methods ensure that list elements are accessed only within valid limits.

Expected Output Achieved

Index error resolved

Safe list access logic implemented

AI suggestion using length checks or exception handling



```
numbers = [1,2,3]
print(numbers[0]) # Changed to access a valid index

...
-----
IndexError                                Traceback (most recent call last)
/tmp/.ipython-input-486182581.py in <cell line: 0>()
      1 numbers = [1,2,3]
----> 2 print(numbers[5])

IndexError: list index out of range

Next steps: Explain error
```

Please explain this error:

`IndexError: list index out of range`

The `IndexError: list index out of range` happens because you're trying to access an element at an index (5) that doesn't exist in the `numbers` list. The valid indices for this list are 0, 1, and 2. To fix this, you need to access an index that is within the valid range of the list. For example, if you want to print the first element, you would use `numbers[0]`.